단원 05

Matplotlib를 활용한 데이터 시각화 기초

인공지능소프트웨어학과

강환수 교수

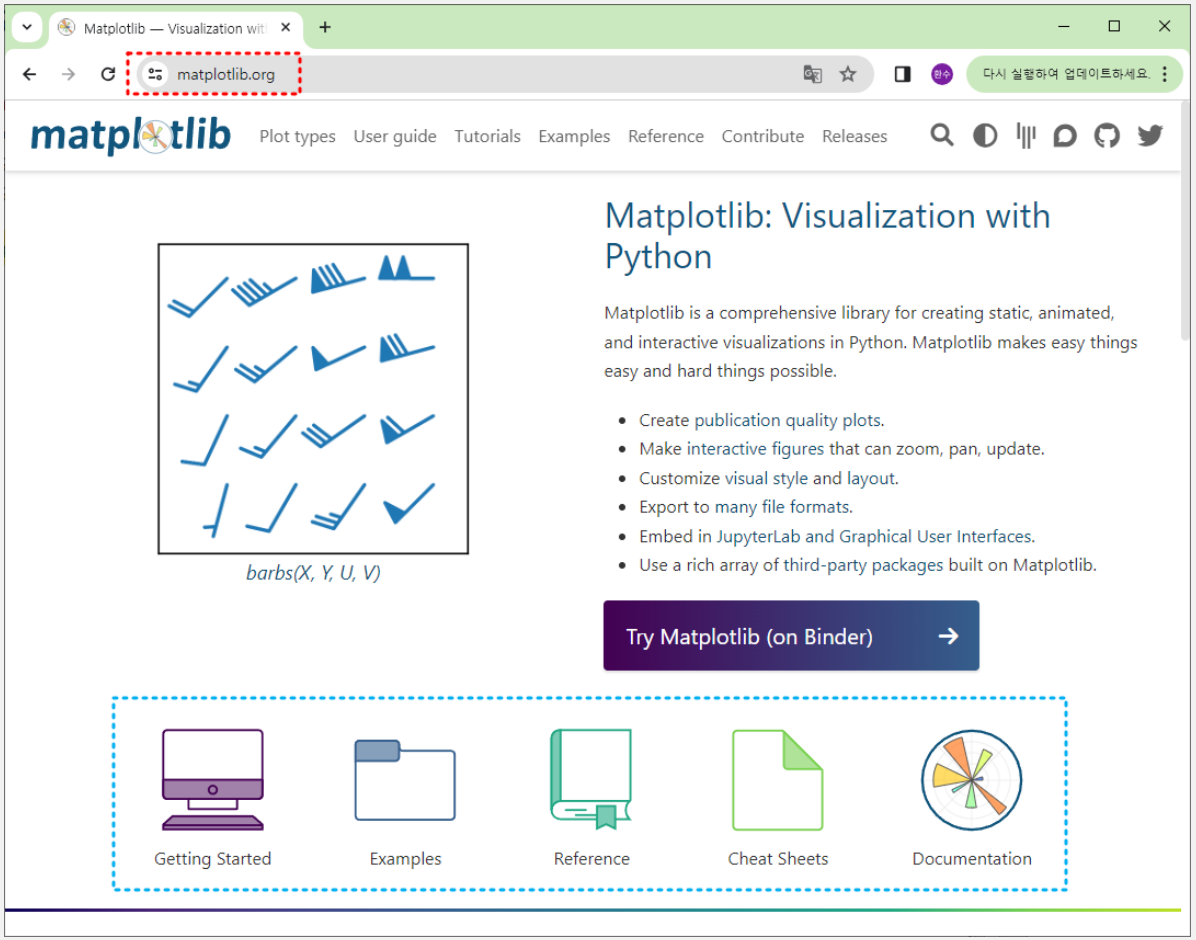
5.1 데이터 시각화 기초

-

데이터 시각화 패키지 matplotlib 개요

2D 그래픽 그림을 생성하는 데 사용되는 데이터 시각화 라이브러리

- 과학 및 엔지니어링 분야에서 데이터를 시각적으로 탐색하고 표현하는 데 널리 사용
- 다양한 차트, 플롯, 히스토그램 등을 생성



설치 확인과 한글 처리 준비

- 그림 고화질 선명도 설정
  - %config InlineBackend.figure_format = 'retina'
- 한글 처리 준비가 플랫폼에 따라 다름
  - Colab 노트북 준비
    - 간단히 한글을 위한 모듈 koreanize-matplotlib 필요
      - !pip install koreanize-matplotlib
      - import koreanize_matplotlib
  - 클라이언트(PC) 노트북 준비, 다음 중 하나 설정
    - 방법 1(함수 rc()로 실행 설정)
      - import matplotlib.pyplot as plt
      - plt.rc('font', family = 'Malgun Gothic')
      - plt.rc('axes', unicode_minus = False)
    - 방법 2(사전 rcParams[]로 실행 설정)
      - import matplotlib.pyplot as plt
      - plt.rcParams['font.family'] = 'Malgun Gothic'
      - plt.rcParams['axes.unicode_minus'] = False

한글 처리

- 클라이언트(PC)의 주피터 노트북에서 한글 처리

```python
import matplotlib.pyplot as plt

# 방법 1: 함수 rc()로 설정
# rc(runtime configuration)
plt.rc('font', family='Malgun Gothic')
plt.rc('axes', unicode_minus=False)
```

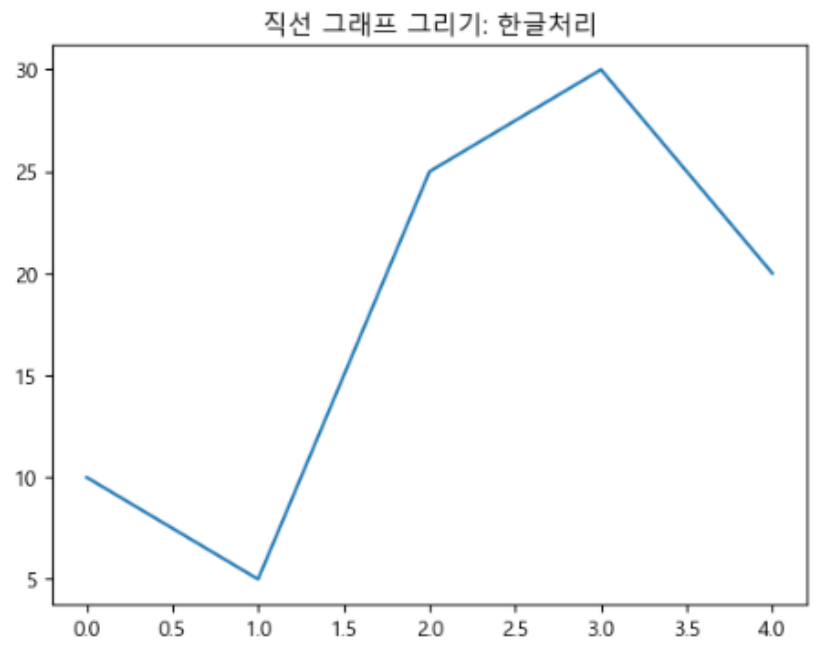
✓ 0.0s

```python
import matplotlib.pyplot as plt

# 방법 2: 사전 rcParams[]에 설정
plt.rcParams['font.family'] ='Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False
```

✓ 0.0s

```python
import matplotlib.pyplot as plt

plt.plot([10, 5, 25, 30, 20])
plt.title('직선 그래프 그리기: 한글처리')
plt.show()
```

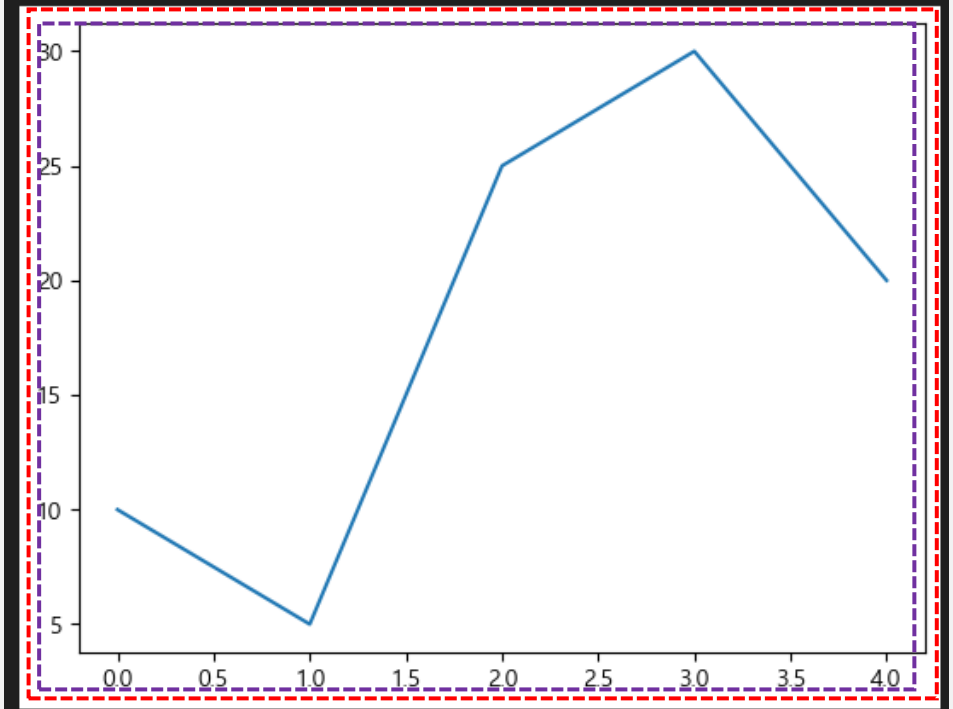
✓ 0.1s



plt.plot(y_value)

- 모듈 matplotlib.pyplot
  - 간단하고 빠른 방식으로 그래프를 그릴 수 있도록 도와주는 모듈
    - 폴더 matplotlib 하부의 pypolt.py
  - 내부적으로 Figure와 Axes 객체를 자동으로 생성하고 관리
    - 간단한 작업에 적합
      - 자동으로 axes를 생성하고 사용
      - 직접 axes 객체를 다룰 필요가 없음
      - 한 두 개의 간단한 그래프를 그릴 때 매우 유용
    - 복잡한 그래프 레이아웃을 구성할 때는 유연성이 떨어질 수 있음

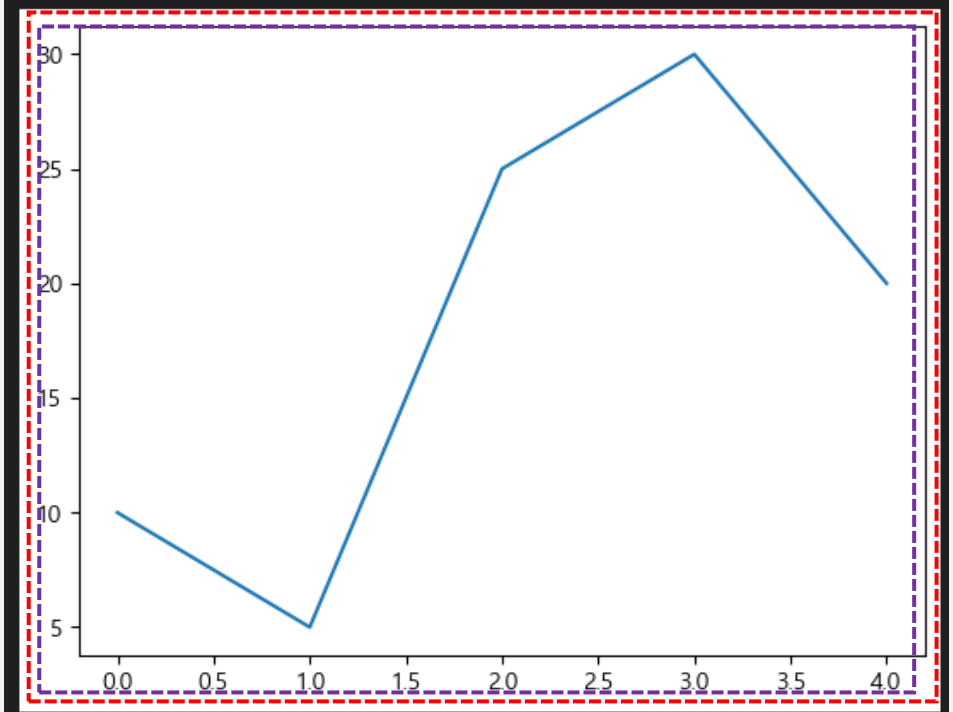
```python
import matplotlib.pyplot as plt

plt.plot([10, 5, 25, 30, 20])
plt.show()
```



Figure와 Axes

- **Figure와 Axes**
  - Figure
    - 전체 그래프가 그려질 "캔버스"
  - Axes
    - 실제 데이터를 시각화하는 영역을 의미
      - 축 x, y와 그림이 그려지는 영역
- **plt.show()**
  - 일반 파이썬 프로그램에서 필요
  - 노트북에서는 필요 없으나 사용하는 것을 권장

```python
import matplotlib.pyplot as plt

plt.plot([10, 5, 25, 30, 20])
plt.show()
```



plt.plot(x_value, y_value)

plt.rcParams['figure.figsize'] = (5, 3)

- **Matplotlib의 모든 플롯에 적용되는 기본값 딕셔너리**
  - rc는 runtime configuration의 약자
  - matplotlibrc 파일의 인메모리 표현이며, 키-값 쌍으로 구성
- **그림 크기 설정**
  - 셀 마다 설정 방법1
    - plt.figure(figsize=(5, 3))
  - 실행 이후 모든 그림에 설정 방법2

```
plt.rcParams['figure.figsize']
```

```
[6.4, 4.8]
```

```
plt.rcParams['figure.figsize'] = (5, 2)
plt.rcParams['figure.figsize']
```

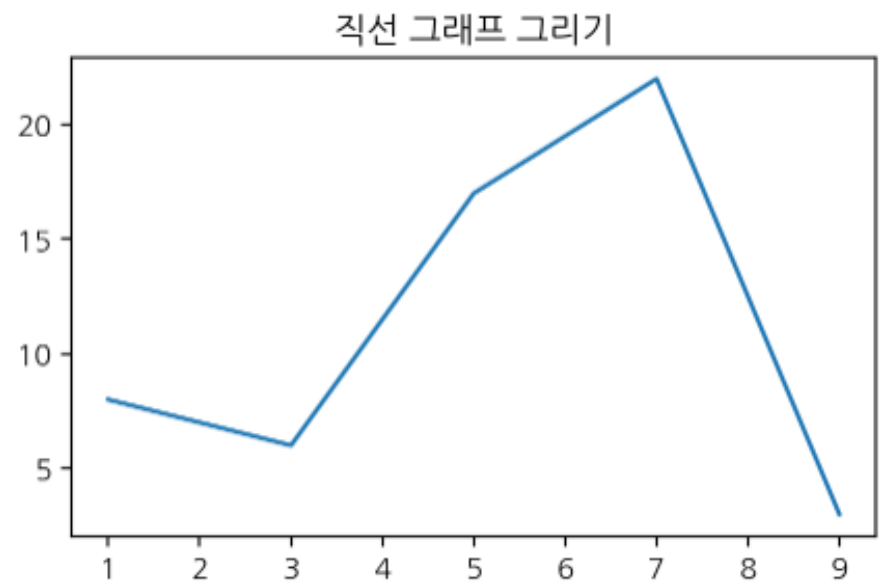
```
[5.0, 2.0]
```

```
plt.rcParams.update(plt.rcParamsDefault)
plt.rcParams['figure.figsize']
```

```
[6.4, 4.8]
```

```
# 모든 피규어의 기본 크기를 5x3으로 설정
plt.rcParams['figure.figsize'] = (5, 3)
```

```
import matplotlib.pyplot as plt

plt.plot([1, 3, 5, 7, 9], [8, 6, 17, 22, 3])
plt.title('직선 그래프 그리기')
plt.show()
```

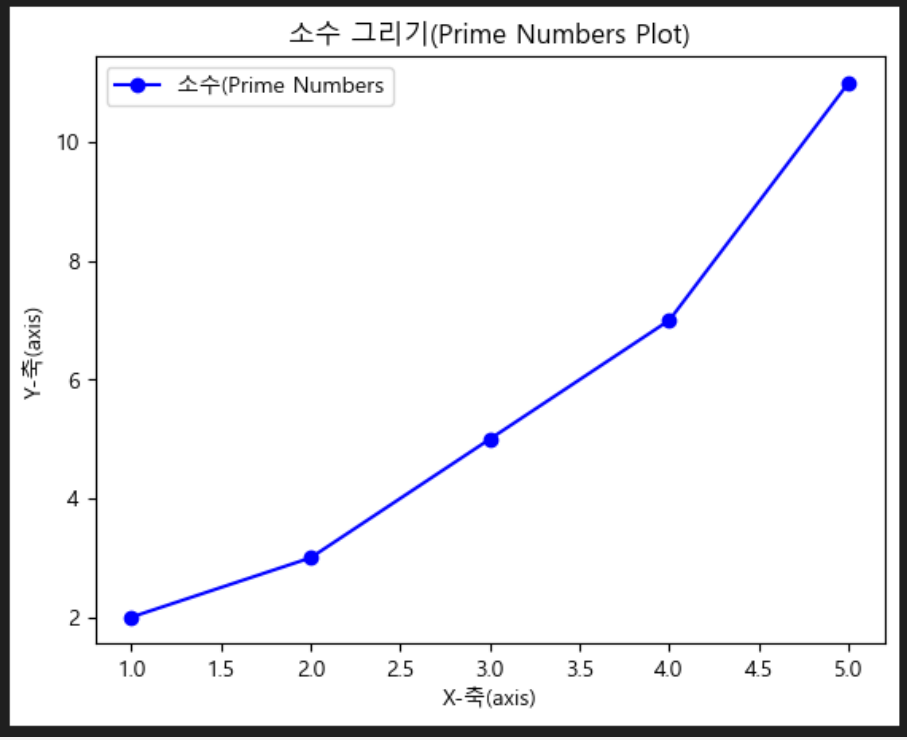


그래프의 여러 속성

- marker
- linestyle
- color
- label

```python
import matplotlib.pyplot as plt

# 데이터 생성
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# 선 플롯 생성
plt.plot(x, y, marker='o', linestyle='-', color='b', label='소수(Prime Numbers')

# 축 및 제목 설정
plt.xlabel('X-축(axis)')
plt.ylabel('Y-축(axis)')
plt.title('소수 그리기(Prime Numbers Plot)')
plt.legend()

# 플롯 표시
plt.show()
```

✓ 0.1s Python

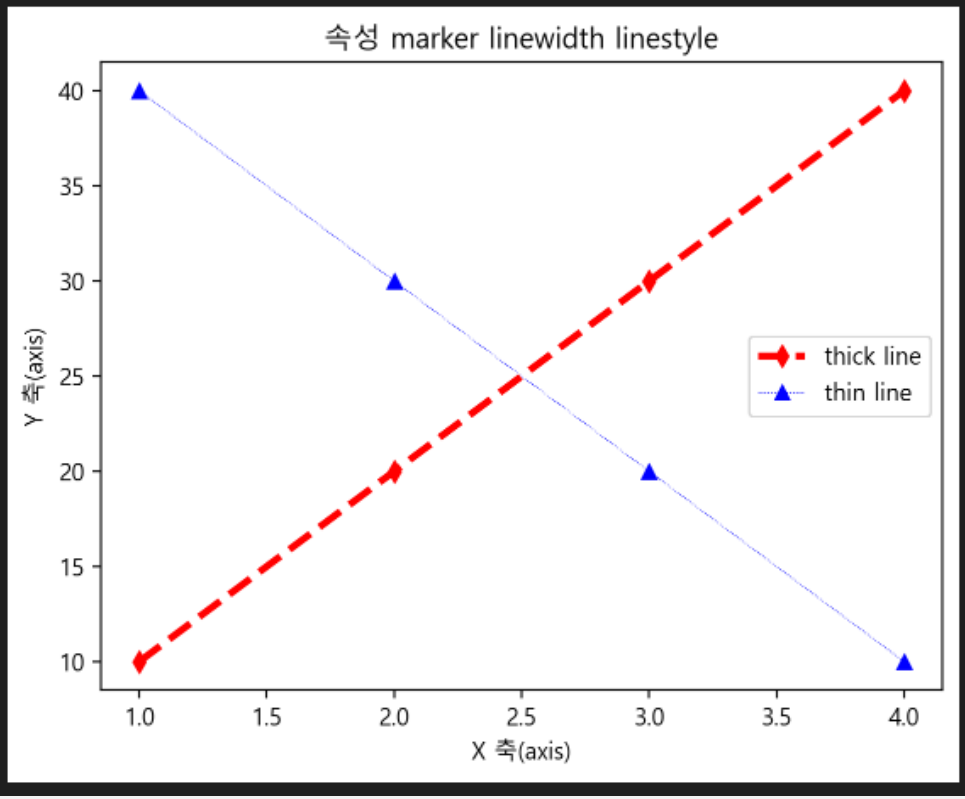


속성 marker, linewidth, linestyle, label

label과 함수 legend()

```python
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [10, 20, 30, 40], color='r', linestyle='--', marker='d', linewidth=3, label='thick line')
plt.plot([1, 2, 3, 4], [40, 30, 20, 10], 'b', ls=':', marker='^', lw=0.5, label='thin line')

plt.title('속성 marker linewidth linestyle')
plt.xlabel('X 축(axis)')
plt.ylabel('Y 축(axis)')
plt.legend()
plt.show()
```

Python

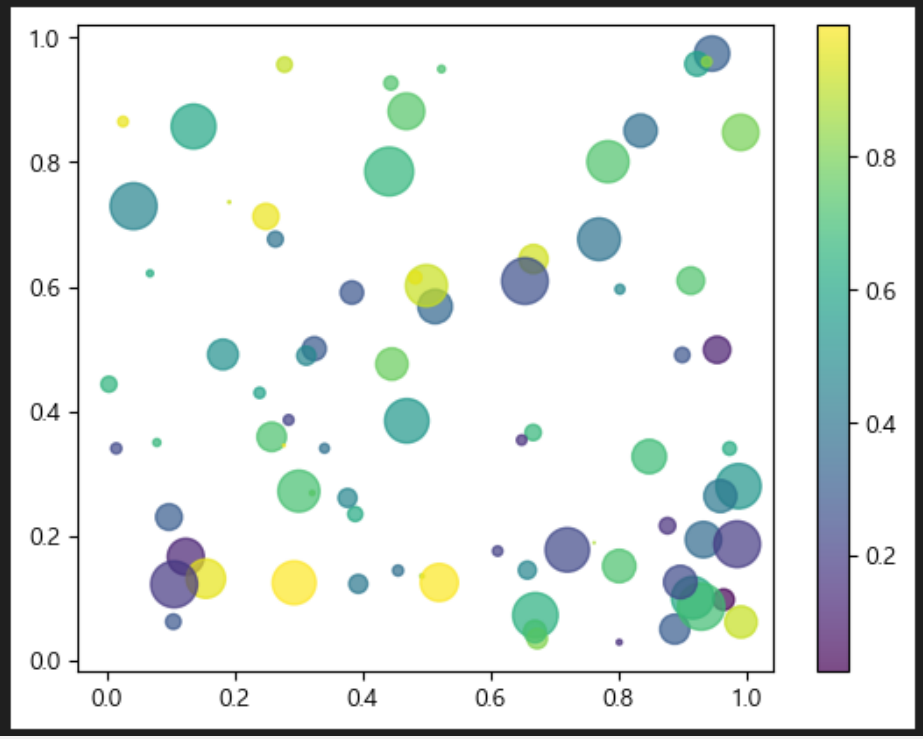


산점도

- 함수 plt.scatter(x, y)
  - 좌표 부분에 속성 marker 유형을 찍는 산점도
  - 마커 기본은 o로 작은 원
  - 인자 s
    - 크기(size)
  - c
    - 색상(color)
  - alpha
    - 불투명도
      - 1이면 정상 색상
      - 작을수록 더 투명해짐
- 난수
  - np.random.rand()
    - [0, 1) 사이의 난수 발생

```python
import numpy as np
import matplotlib.pyplot as plt

# Fixing random state for reproducibility
np.random.seed(2025)

N = 80
x = np.random.rand(N)
y = np.random.rand(N)
colors = np.random.rand(N)
area = (20 * np.random.rand(N)) ** 2  # 0 to 15 point radii

plt.scatter(x, y, s=area, c=colors, alpha=0.7)
plt.colorbar()
plt.show()
```

✓ 0.1s Python



논리 인덱싱(Boolean indexing)

- **np.random.randint(start, stop, size)**
  - [start, stop)에서 size의 난수(random number) 생성
- **논리 인덱싱**
  - 논리 값이 True인 원소만 추출

```python
import numpy as np

x = np.random.randint(-10, 10, 10) # [-10, 10) 중에서 10개의 랜덤 값 추출

print(x)        # 10개 난수 목록
print(x > 0)    # 논리 연산
print(x[x > 0]) # 논리 인덱싱(boolean indexing)
```

✓ 0.0s Python

```
[-8 -4  5  1 -1 -9  4 -8  2  5]
[False False  True  True False False  True False  True  True]
[5 1 4 2 5]
```

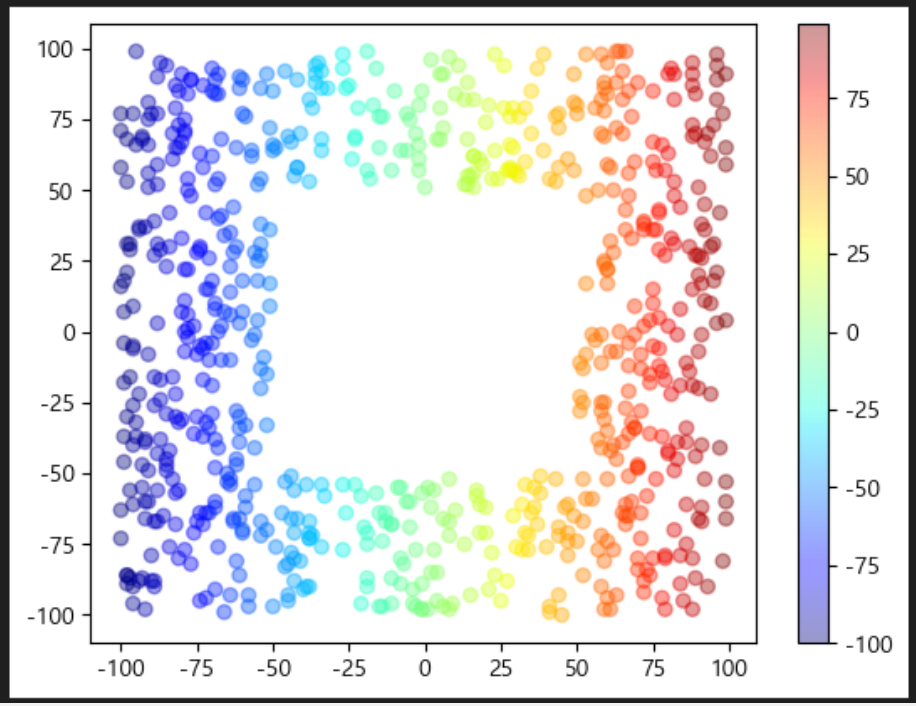
산점도

논리 색인(boolean index)를 만들어 모든 좌표 x와 y에서 mask1 또는 mask2를 만족하는 좌표 추출

- **연산자 +**
  - 논리 or 연산

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.random.randint(-100, 100, 1000) # 1000개의 랜덤 값 추출
y = np.random.randint(-100, 100, 1000) # 1000개의 랜덤 값 추출

mask1 = abs(x) > 50 # x에 저장된 값 중 절댓값이 50보다 큰 값 걸러 냄
mask2 = abs(y) > 50 # y에 저장된 값 중 절댓값이 50보다 큰 값 걸러 냄
x = x[mask1 + mask2] # mask1과 mask2 중 하나라도 만족하는 값 저장
y = y[mask1 + mask2] # mask1과 mask2 중 하나라도 만족하는 값 저장

plt.scatter(x, y, c=x, cmap='jet', alpha = 0.4)
plt.colorbar()
plt.show()
```

✓ 0.1s Python

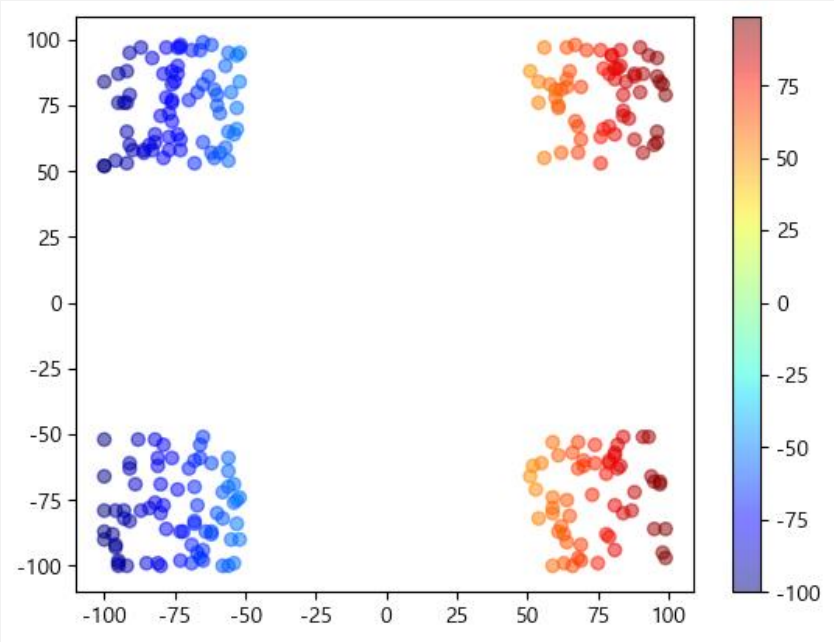


산점도

논리 색인(boolean index)를 만들어 모든 좌표 x와 y에서 mask1와 mask2를 만족하는 좌표 추출

- **연산자 ***
  - 논리 and 연산

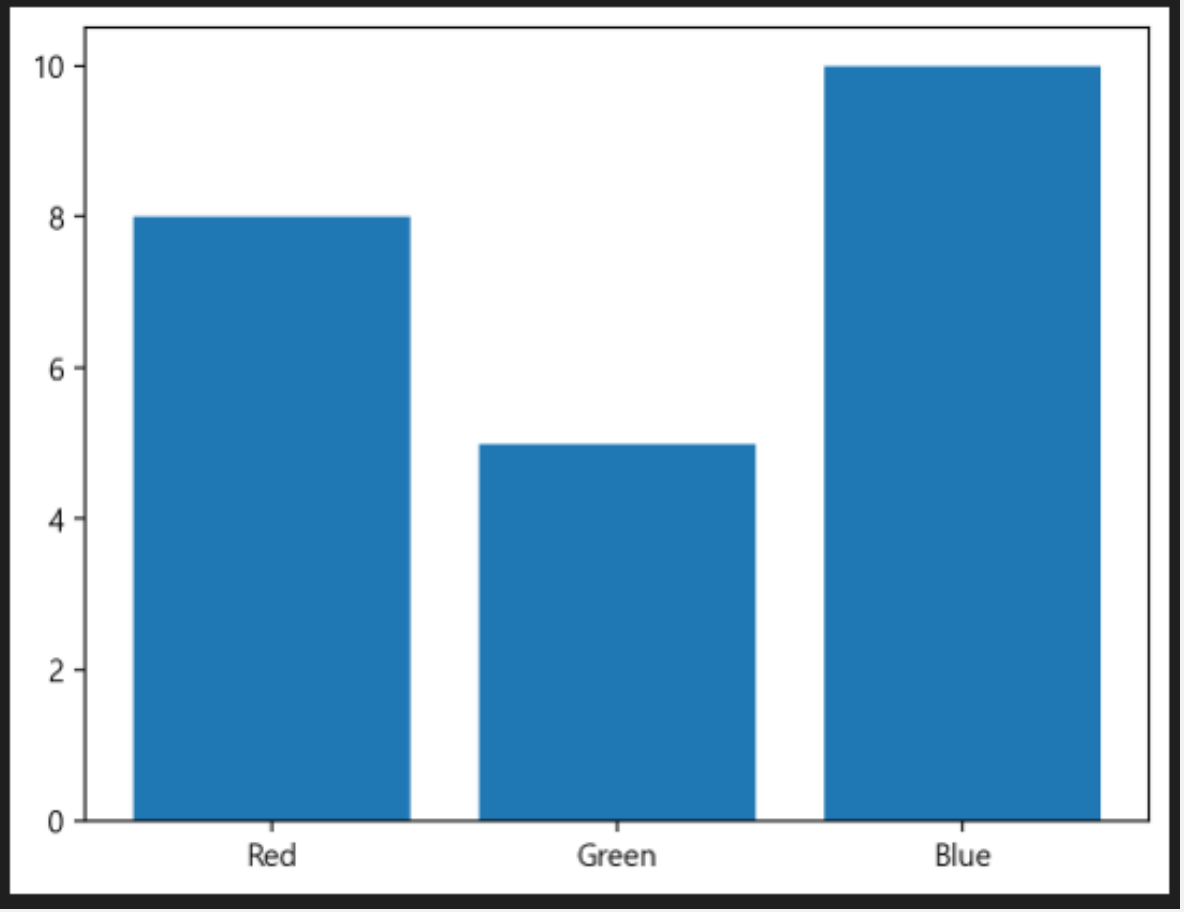
```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.randint(-100, 100, 1000) # 1000개의 랜덤 값 추출
y = np.random.randint(-100, 100, 1000) # 1000개의 랜덤 값 추출

mask1 = abs(x) > 50 # x에 저장된 값 중 절댓값이 50보다 큰 값 걸러 냄
mask2 = abs(y) > 50 # y에 저장된 값 중 절댓값이 50보다 큰 값 걸러 냄
x = x[mask1 * mask2] # mask1과 mask2 모두 만족하는 값 저장
y = y[mask1 * mask2] # mask1과 mask2 모두 하나라도 만족하는 값 저장
# x = x[mask1 & mask2] # mask1과 mask2 모두 만족하는 값 저장
# y = y[mask1 & mask2] # mask1과 mask2 모두 하나라도 만족하는 값 저장

plt.scatter(x, y, c=x, cmap='jet', alpha = 0.5)
plt.colorbar()
plt.show()
```



막대 그래프

- **plt.bar()**

```
import matplotlib.pyplot as plt

plt.bar(["Red", "Green", "Blue"], [8, 5, 10])
plt.show()
```

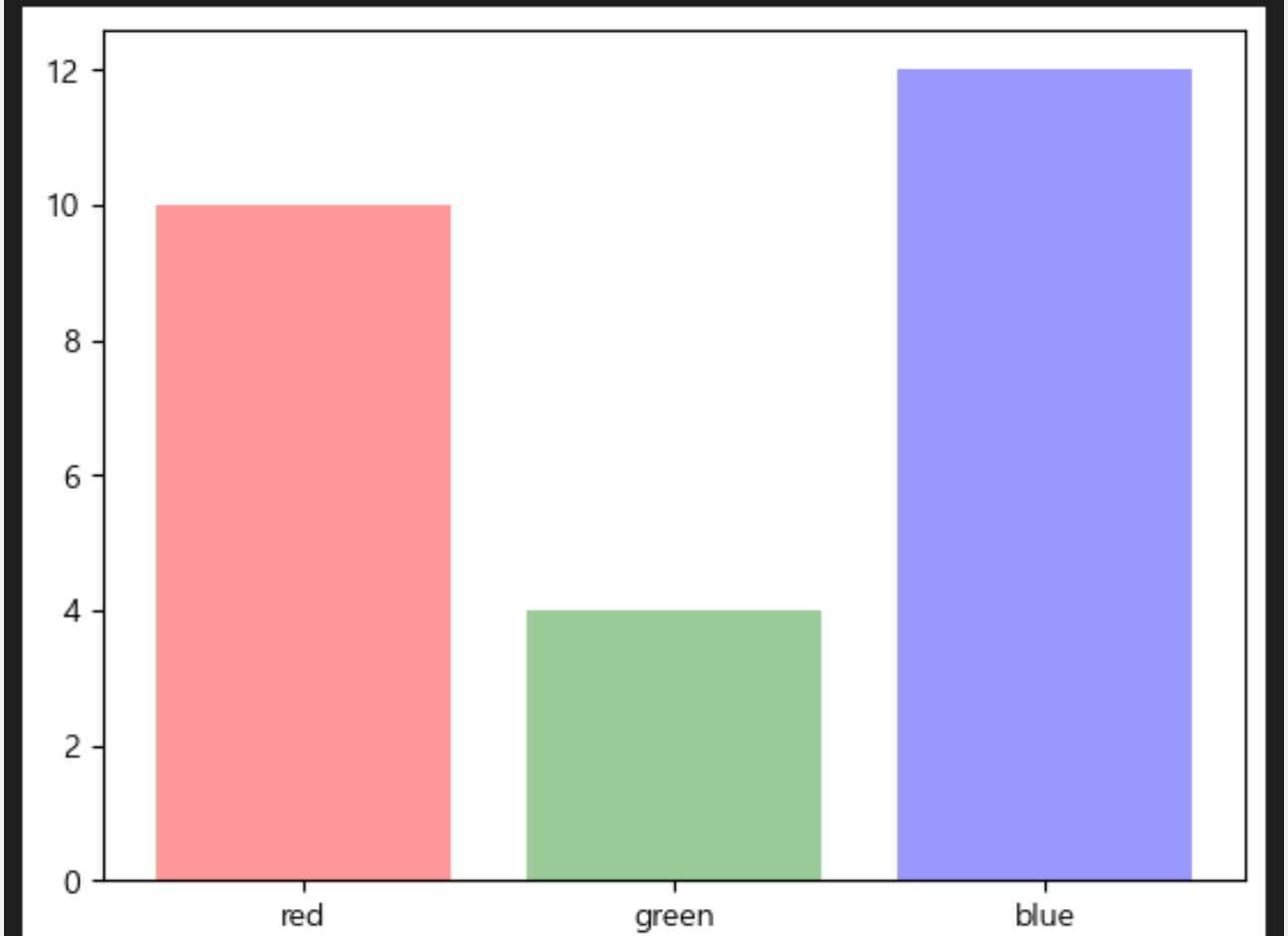


속성 color

- 색상 지정

```
import matplotlib.pyplot as plt

c = ["red", "green", "blue"]
plt.bar(c, [10, 4, 12], color=c, alpha=.4)
plt.show()
```

✓ 0.0s



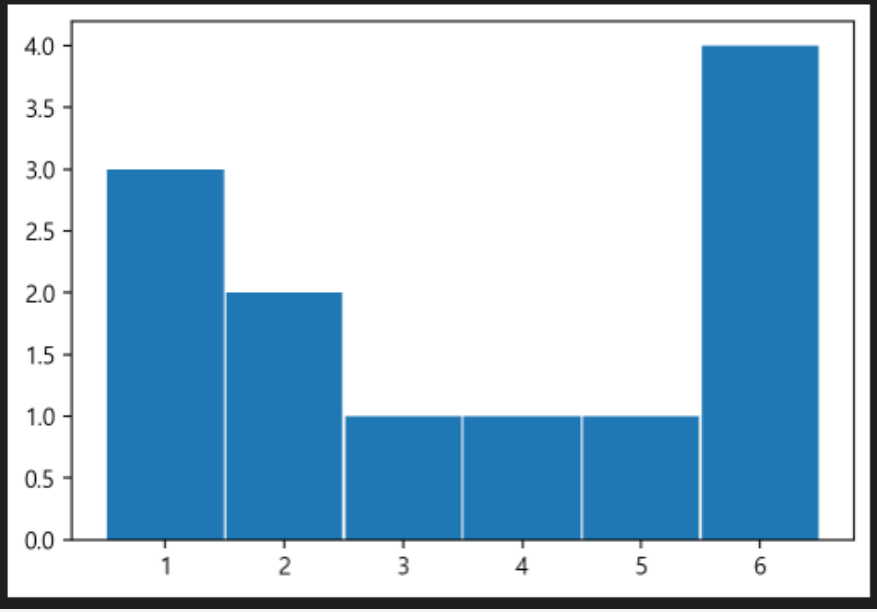
돗수분포표 그림 hist()

데이터의 분포를 시각화하는 데 유용, 데이터가 특정 구간에서 얼마나 자주 발생하는지(빈도)를 나타내는 막대그래프

- **히스토그램 구간 bins**
  - 구간(bin)의 개수 또는 구간의 경계를 지정
    - 정수 값: 구간의 개수를 지정, 기본은 10개
    - 리스트나 배열: 구간의 경계를 명시적으로 지정

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4)) # 그림 크기 지정

plt.hist([1, 1, 1, 2, 2, 3, 4, 5, 6, 6, 6, 6], rwidth=.98, bins=[.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5])
plt.xticks(range(1, 7))
plt.show()
```

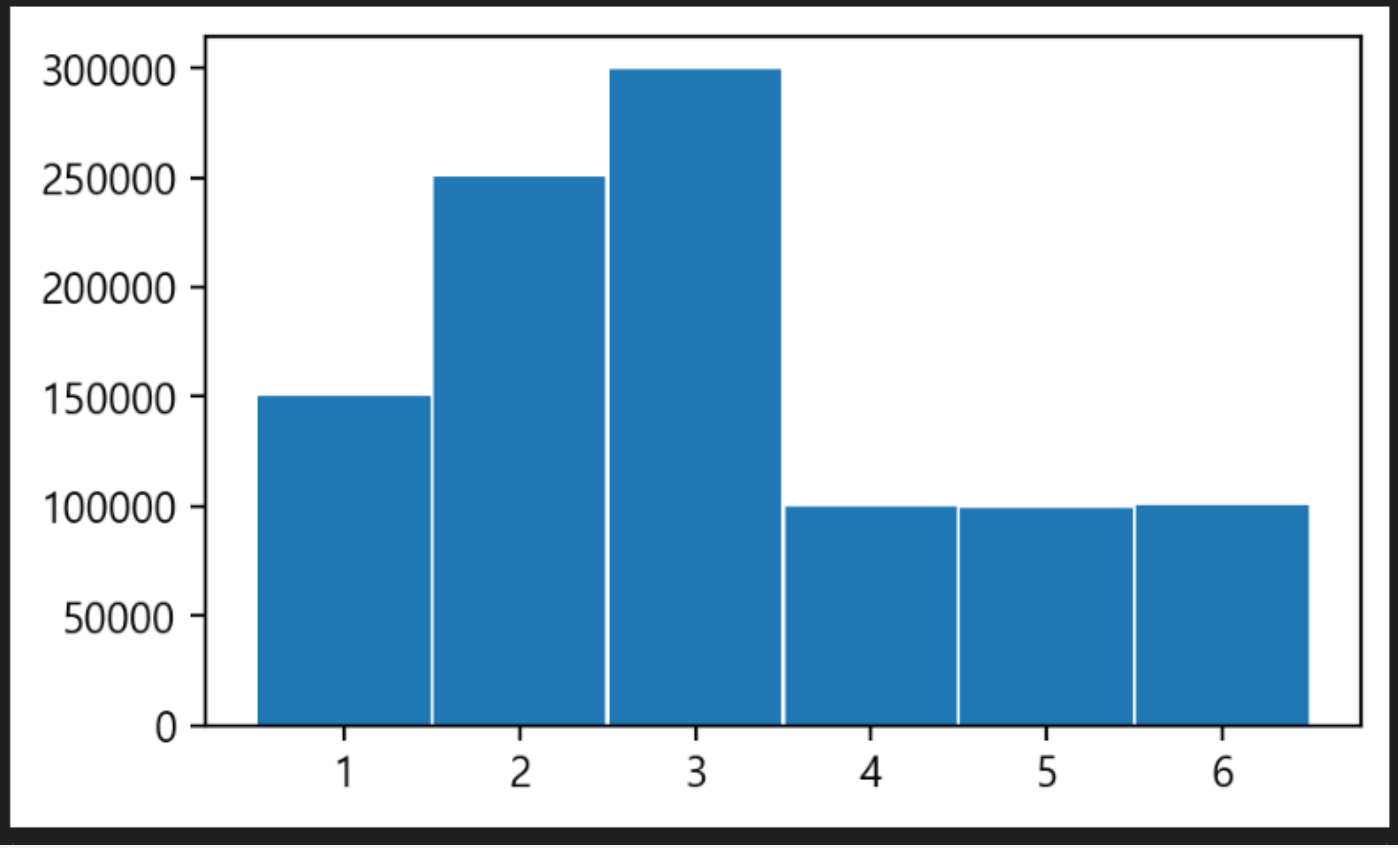
Python



주사위 모의 실험 도수분포표

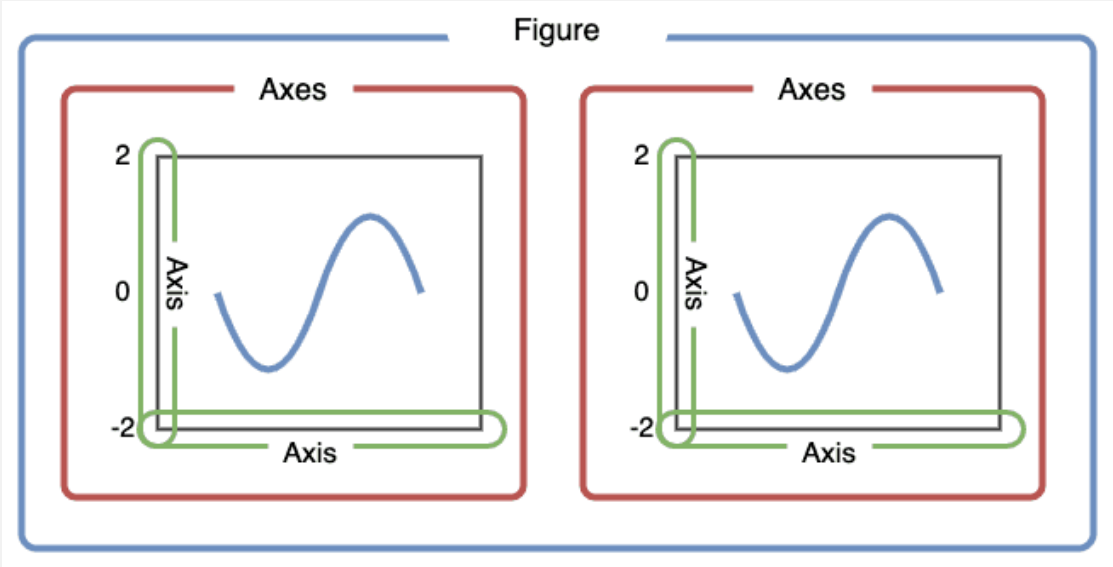
```python
import matplotlib.pyplot as plt
import numpy as np

dice = np.random.choice(range(1, 7), 1000000,
                        p = [0.15, 0.25, 0.3, 0.1, 0.1, 0.1])
plt.figure(figsize=(5, 3), dpi=200)

plt.hist(dice, rwidth=.98, bins=np.arange(.5, 7))
plt.xticks(range(1, 7))

plt.savefig('주사위.png')
plt.show()
```

✓ 0.2s Python

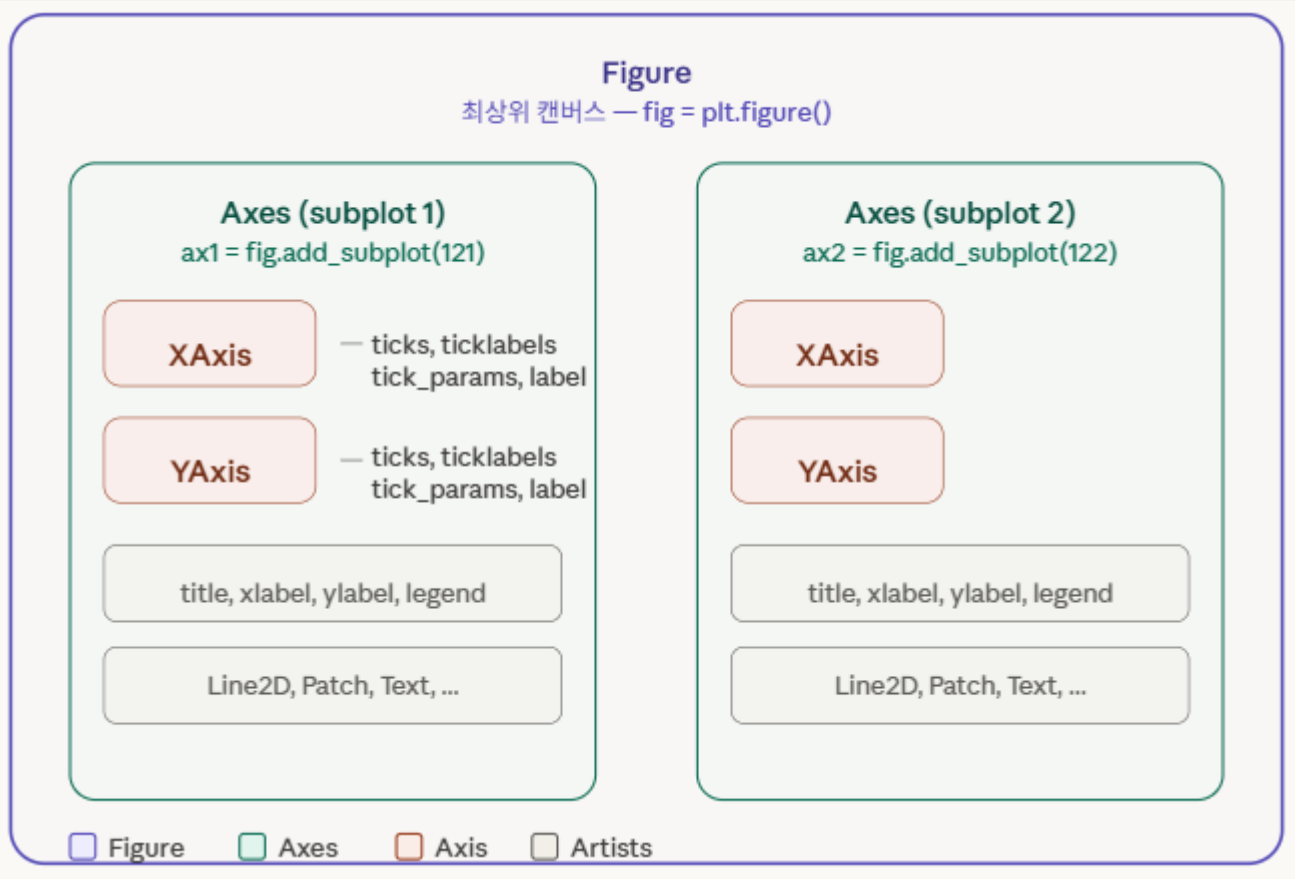


Figure와 Axes 개요

- **전체 그림의 바탕인 Figure**
  - 그래프를 담는 캔버스나 종이 시트
  - 하나의 Figure 객체에는 하나 이상의 부분그림(subplot)인 Axes 객체가 포함 가능
  - ● Figure 객체: 전체 캔버스
  - ● Axes 객체(object):
    - Figure 객체에 속하며 시각화할 데이터를 추가하는 공간
    - Figure 내부에서 실제 그래프가 그려지는 영역
  - ● Axis 객체:
    - Axes 객체에 속하는 축을 말하며, Axis는 다시 X Axis, Y Axis로 분류



Figure와 Axes, axis



import matplotlib.pyplot as plt

- 하나의 Figure에 여러 개의 그래프를 동시에 그리는 레이아웃
  - 서브플롯과 “부분 그림”이란 용어를 혼용해서 쓸 예정
- 서브플롯을 그리는 방법
  - import matplotlib.pyplot as plt
    - plt는 패키지 matplotlib 하부에 있는 모듈 이름
    - plt 역할
      - 내부적으로 Figure와 Axes 객체를 자동으로 생성하고 관리

서브플롯 개념과 서브플롯을 그리는 다양한 방법

- **plt 함수**
  - plt.subplot(m, n, index): 반환 값: axes
    - 현재 그림에 세부 그림 축(axes)을 하나 추가하거나 기존 축을 참조
      - Add an Axes to the current figure or retrieve an existing Axes.
  - plt.subplots(m, n): 반환 값이 2개
    - Fig: Figure, Ax: Axes or array of Axes
    - 캔버스 figure에 지정한 일련의 subplots을 생성
      - Create a figure and a set of subplots.
- **figure 메소드**
  - fig.add_subplot(m, n, index), 반환 값: axes
    - 하위 플롯 구성의 일부로 그림에 하나의 Axes을 추가
      - Add an Axes to the figure as part of a subplot arrangement.

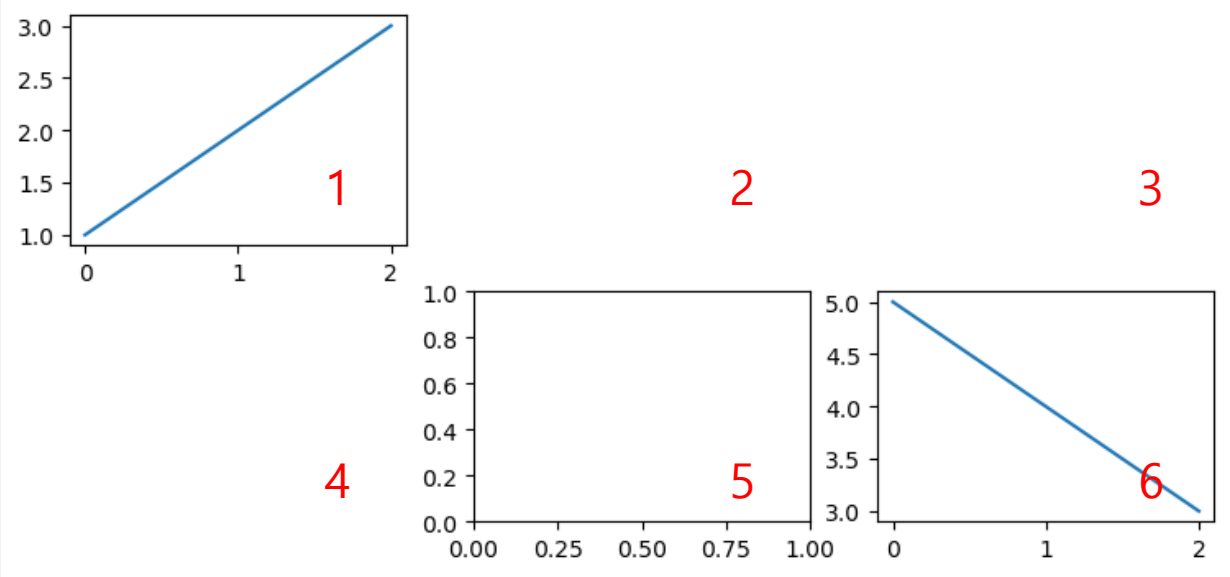
plt.subplot(m, n, index)

현재 그림에 index 위치에 세부 그림 축(axes)을 하나 추가

- **subplot(nrows, ncols, index)의 형식**
  - nrows: 플롯의 행(row) 개수
  - ncols: 플롯의 열(column) 개수
  - index: 현재 활성화할 플롯의 위치(1부터 시작)
- **특징**
  - 개별적으로 각 서브플롯을 지정하는 방식
    - 하나의 플롯을 설정한 후 바로 그 다음 플롯을 설정할 때 유용
  - 다양한 플롯 레이아웃을 쉽게 조작할 수 있는 유연성을 제공
    - 복잡한 다중 플롯 레이아웃에서는 코드가 길어지고 가독성이 떨어질 수 있음

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(9, 4))

plt.subplot(2, 3, 1)
plt.plot([1, 2, 3])
plt.subplot(2, 3, 5)
plt.subplot(2, 3, 6)
plt.plot([5, 4, 3])

plt.show()
```

바로 plt.plot()으로 해당 부분 플롯에 그림을 그림



plt.subplots()

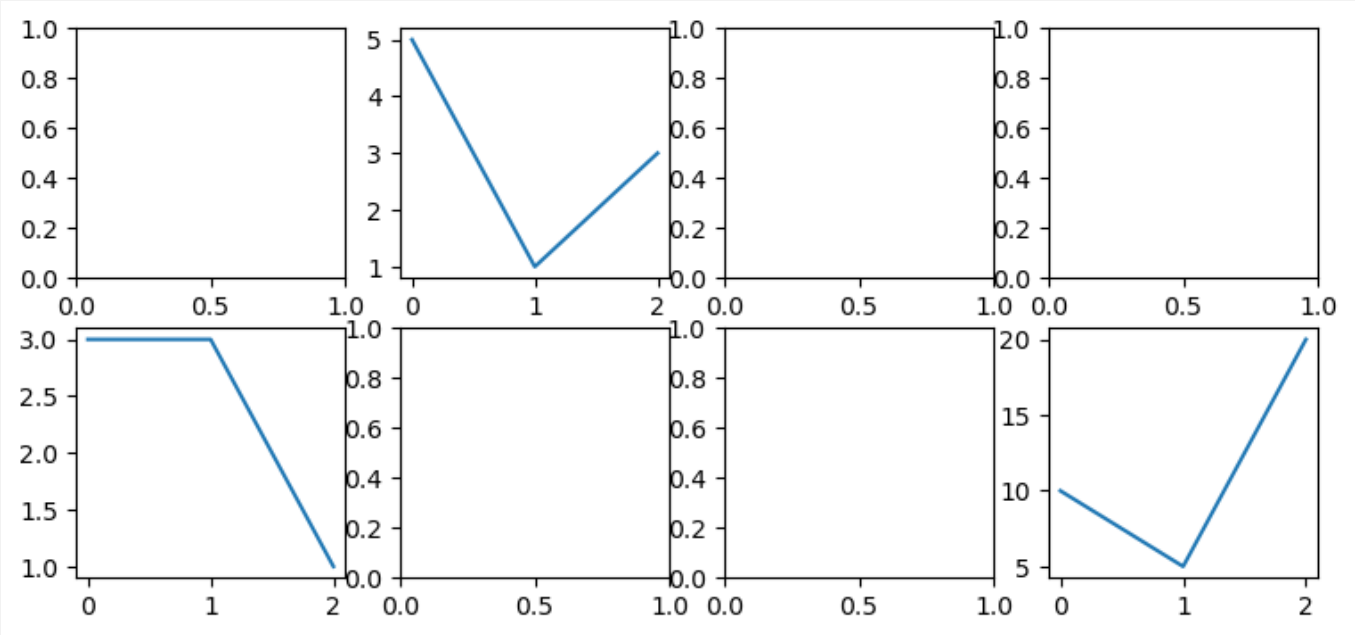
도화지 figure에 지정한 subplots을 생성

- **subplots(nrows, ncols)의 형식**
  - nrows
    - 플롯의 행(row) 개수
  - ncols
    - 플롯의 열(column) 개수
- **특징**
  - 한 번의 호출로 여러 개의 Axes 객체를 한꺼번에 생성하여 반환
  - fig와 axs 두 개의 객체를 반환
    - fig
      - Figure 객체
    - axs
      - 여러 개의 Axes 객체 배열 (또는 단일 Axes 객체)
  - 더 직관적이고 효율적으로 다중 플롯을 그릴 수 있게 해 줌
    - 배열 형태로 반환된 Axes 객체를 통해 각각의 플롯에 접근

```python
import matplotlib.pyplot as plt

# plt.figure(figsize=(9, 4)) # 반영이 안됨

fig, axs = plt.subplots(2, 4) # 2행 4열의 서브플롯
fig.set_size_inches((9, 4)) # 캔버스의 크기 지정

axs[0, 1].plot([5, 1, 3])
axs[1, 0].plot([3, 3, 1])
plt.plot([10, 5, 20])
plt.show()
```

반환 받은 axs에서 첨자로 각각의 그림을 그림

마지막 그림(ax[1, 3])은 plt로도 가능



plt.subplots()에서 캔버스 figure의 크기 지정 주의점

- **plt.subplots()**
  - figure와 그 안의 Axes 객체들을 한 번에 생성하는 함수
- **다음 코드의 문제**
  - plt.figure(figsize=(9, 4))을 호출
    - 이미 figure가 생성
  - plt.subplots(2, 4)
    - 다시 새로운 figure를 생성
  - 따라서 두 번째 figure가 생성되며, 첫 번째 figure는 사실상 무시

```
import matplotlib.pyplot as plt

plt.figure(figsize=(9, 4)) # 반영이 안됨
fig, axs = plt.subplots(2, 4) # 2행 4열의 서브플롯
# fig.set_size_inches((9, 4))
plt.show()
```

- **다음 2 가지 방법으로 그림 크기 설정**

```
fig, ax = plt.subplots(2, 3, figsize=(7, 4)) # 방법1
fig.set_size_inches(6, 3) # 방법2
```

plt.subplots()에서 캔버스 figure의 크기 지정

- **fig의 크기를 지정하는 방법**
  - 방법1
    - plt.subplots()에서 figsize=(8, 6)로 직접 지정
      - plt.subplots(2, 1, figsize=(8, 6))
  - 방법2
    - plt.subplots()에서 반환 받은 fig에서 메소드 set_size_inches(8, 6)로 지정
      - fig, axes = plt.subplots(2, 1)
      - fig.set_size_inches(10, 4)

```python
# plt.subplots()에서 직접 캔버스의 크기 지정

import matplotlib.pyplot as plt

# plt.subplots()로 figure와 크기를 지정
fig, axes = plt.subplots(2, 1, figsize=(8, 6))

# 각 서브플롯에 데이터 추가
axes[0].plot([1, 2, 3], [1, 4, 9])
axes[1].plot([1, 2, 3], [1, 2, 3])

plt.show()
```

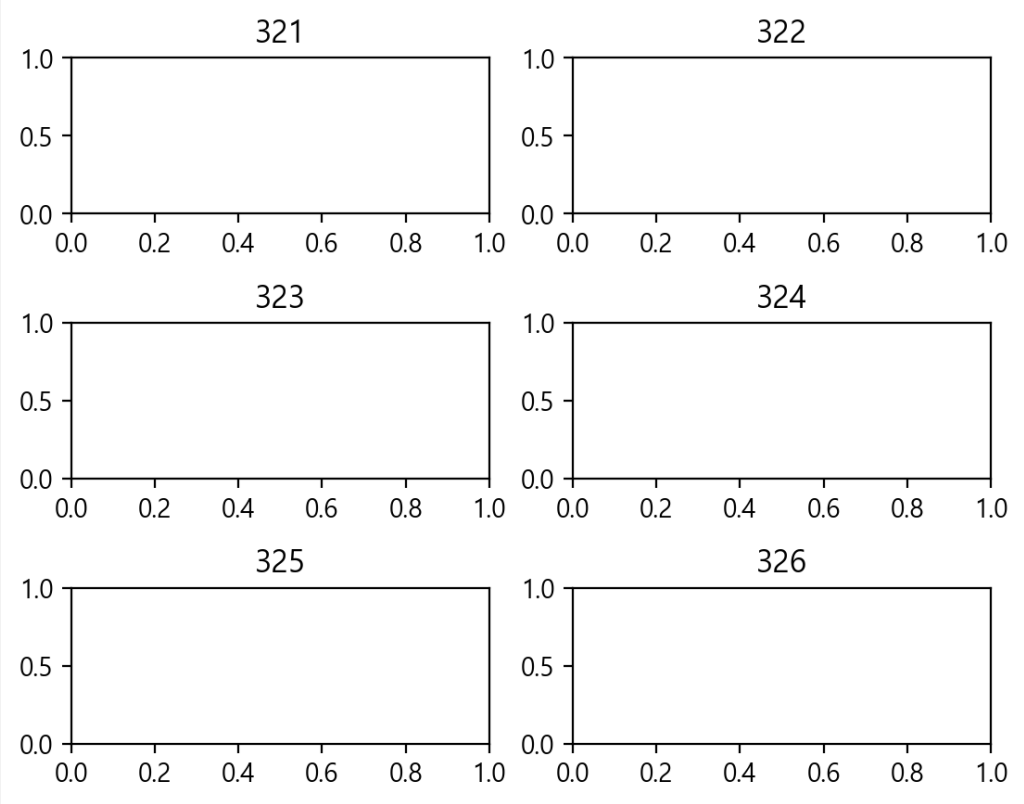
```python
# plt.subplots()에서 반환 받은 fig에서 크기 지정

import matplotlib.pyplot as plt

# plt.subplots()로 figure와 크기를 지정
fig, axes = plt.subplots(2, 1, figsize=(8, 6))
fig.set_size_inches(10, 4)

# 각 서브플롯에 데이터 추가
axes[0].plot([1, 2, 3], [1, 4, 9])
axes[1].plot([1, 2, 3], [1, 2, 3])

plt.show()
```

서브플롯 fig.add_subplot(m, n, index) 개요

하위 플롯 구성의 일부로 그림에 하나의 Axes을 추가

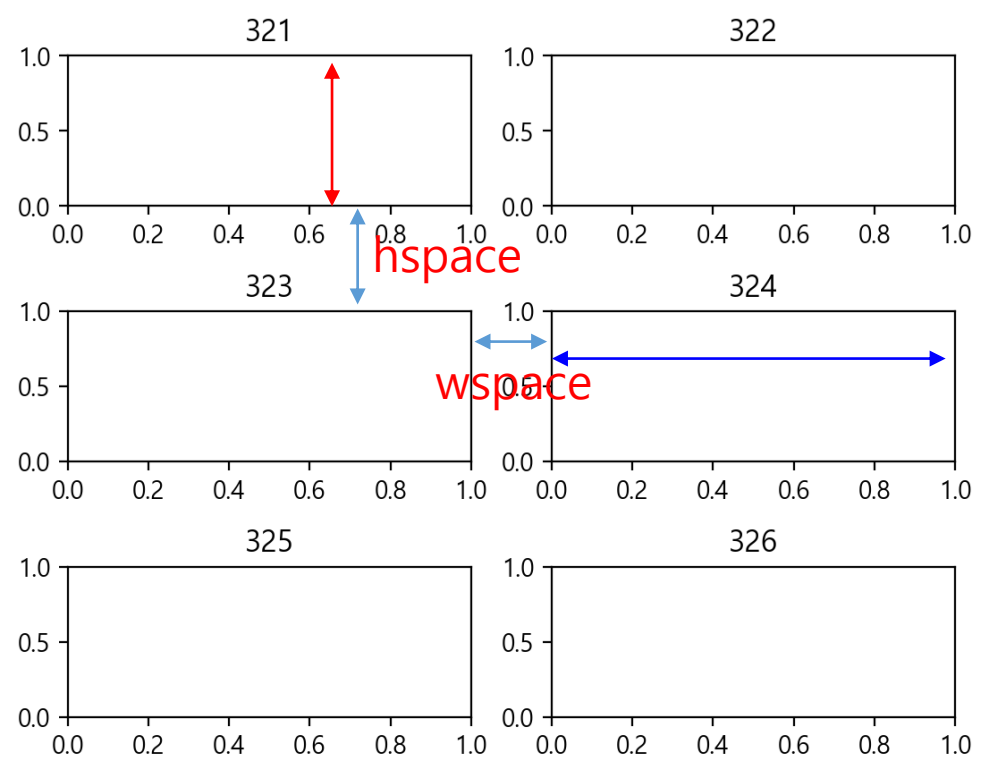
- **fig.add_subplot(m, n, index)으로 부분그림 그리기**
  - 부분그림에서 m은 행의 수, n은 열의 수
  - index는 1부터 시작하는 부분 플롯의 번호
    - index는 행이 우선
  - (3, 2, 1)은 (321)로 쓰는 것이 가능
    - (321)로 사용하는 경우, 단 단위 자리만 가능하므로 1에서 9까지만 가능

```python
import matplotlib.pyplot as plt

# define figure
fig = plt.figure()

# add subplots
fig.add_subplot(321).set_title('321')
fig.add_subplot(322).set_title('322')
fig.add_subplot(323).set_title('323')
fig.add_subplot(324).set_title('324')
fig.add_subplot(325).set_title('325')
fig.add_subplot(326).set_title('326')

# adjust spaces
fig.subplots_adjust(wspace=.2, hspace=.7)
plt.show()
```



서브플롯 사이의 간격 조정

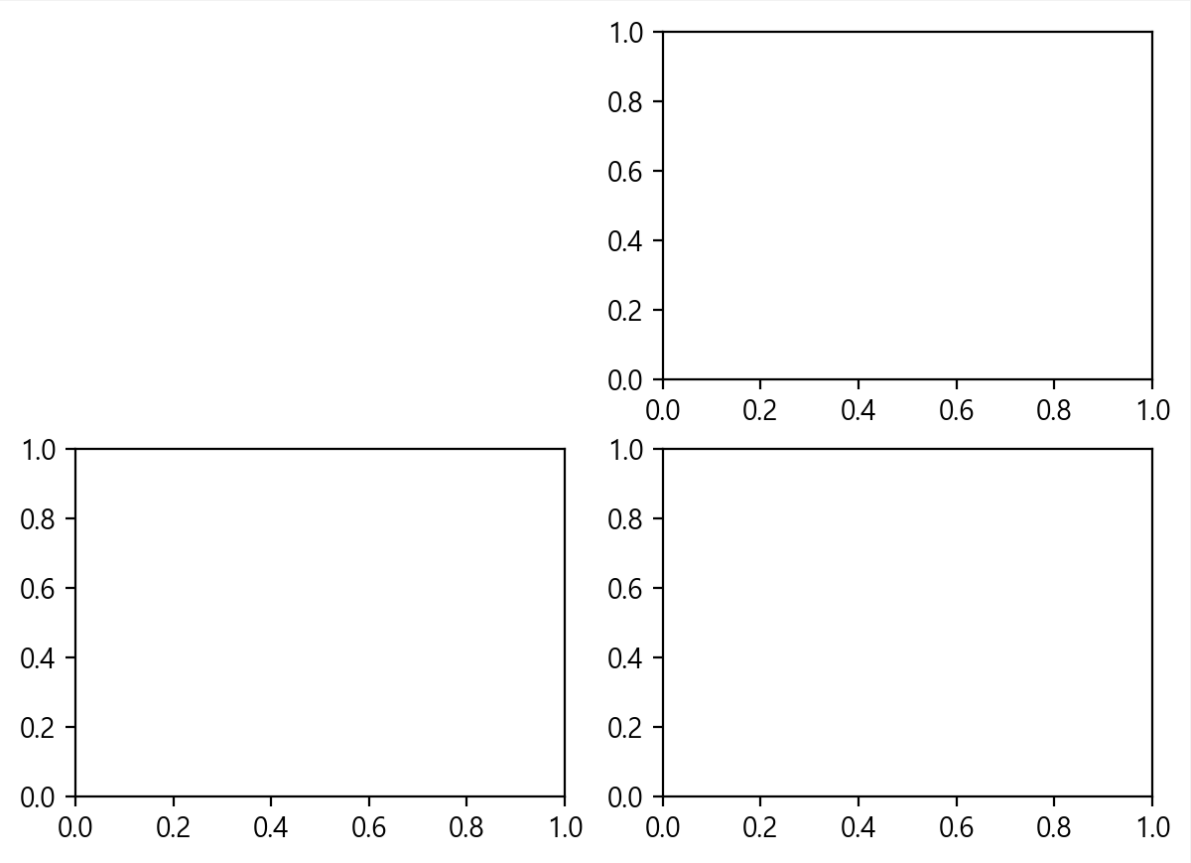
- fig = plt.figure()
- # adjust spaces
- fig.subplots_adjust(wspace=.2, hspace=.7)
  - 인자 wspace
    - 평균 Axes 너비에 대한 분수로 표현된 하위 플롯 간 패딩 너비
  - 인자 hspace
    - 평균 Axes 높이의 분수로 표현된 하위 플롯 간 패딩 높이



서브플롯 fig.add_subplot() 활용

- **반환 값**
  - axes

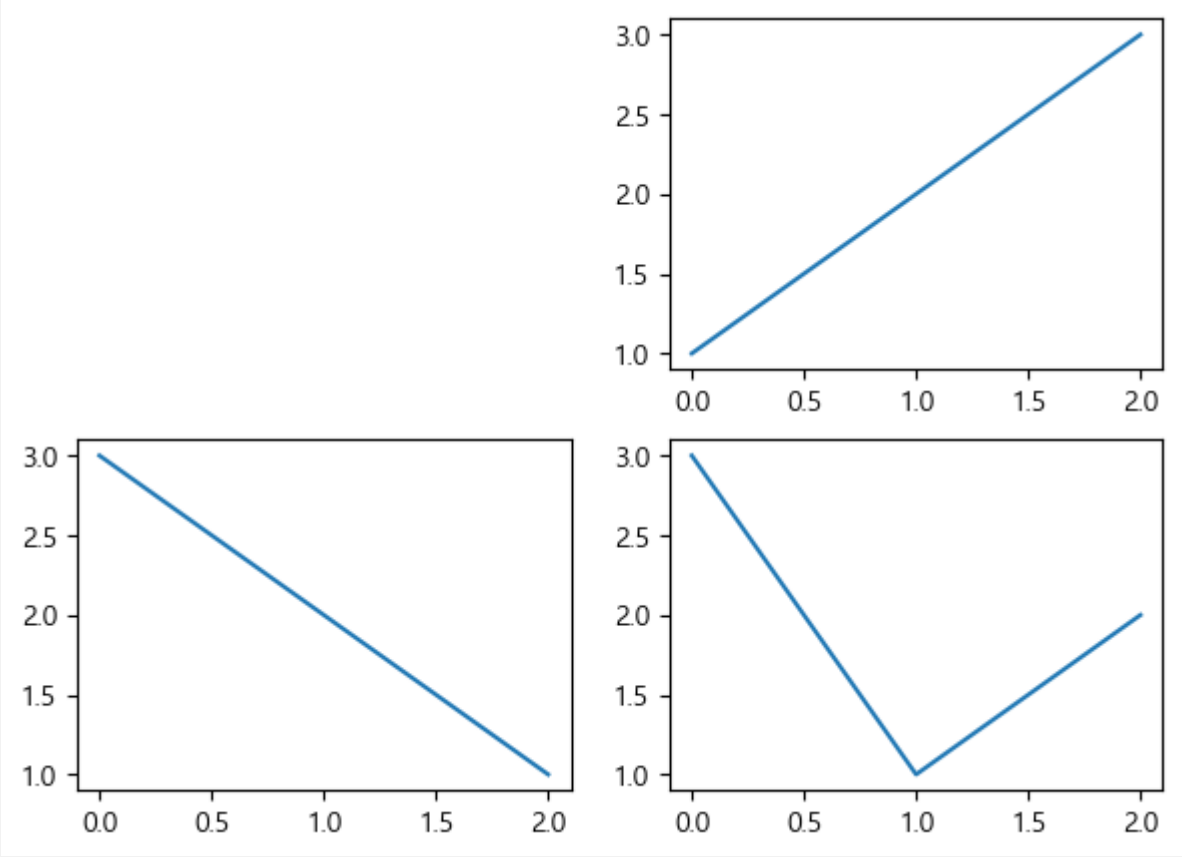
```
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(7, 5))

ax2 = fig.add_subplot(2, 2, 2)
ax3 = fig.add_subplot(2, 2, 3)
ax4 = fig.add_subplot(2, 2, 4)
```



서브플롯 fig.add_subplot() 그리기

- 반환 값 ax로 그리기

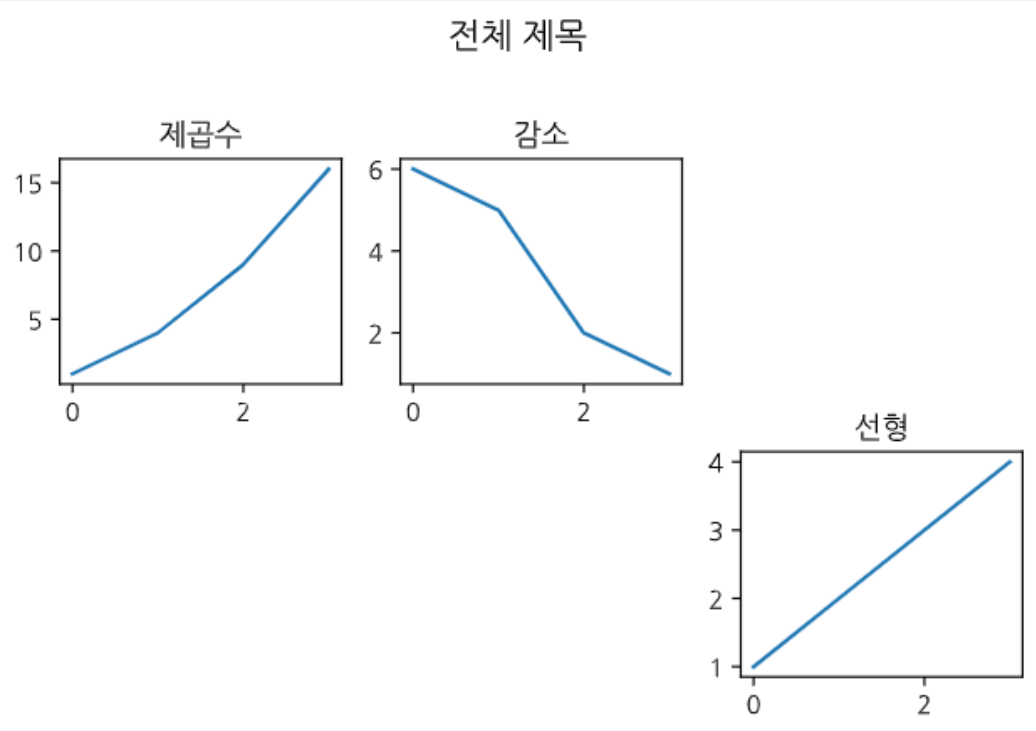
```
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(7, 5))

ax2 = fig.add_subplot(2, 2, 2)
ax3 = fig.add_subplot(2, 2, 3)
ax4 = fig.add_subplot(2, 2, 4)

ax2.plot([1, 2, 3])
ax3.plot([3, 2, 1])
ax4.plot([3, 1, 2]);
```



- 해당 그림은 바로 plt로도 그릴 수 있음

plt.subplot() 그림 제목

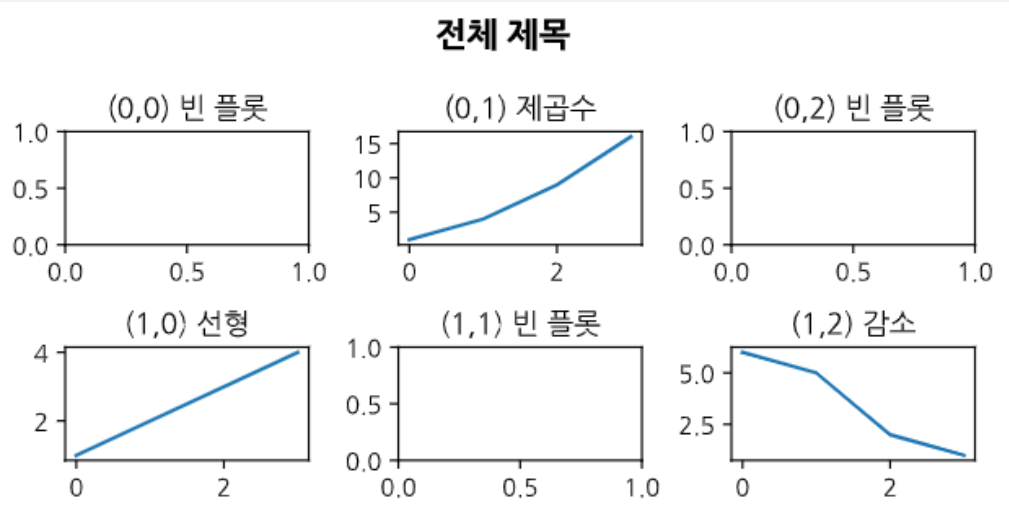
- **plt.suptitle('전체제목')**
- **plt.title('부분제목')**




```
import matplotlib.pyplot as plt

# 전체 제목 – 어느 위치에서 호출해도 무관 (Figure 레벨)
plt.suptitle('전체 제목', fontsize=14, y=1.02)

# subplot 1 – 생성 직후 바로 title 지정
plt.subplot(2, 3, 1)
plt.plot([1, 4, 9, 16])
plt.title('제곱수')

# subplot 5
plt.subplot(2, 3, 6)
plt.plot([1, 2, 3, 4])
plt.title('선형')

# subplot 2
plt.subplot(2, 3, 2)
plt.plot([6, 5, 2, 1])
plt.title('감소')

plt.tight_layout()
plt.show()
```

plt.subplots() 그림 제목

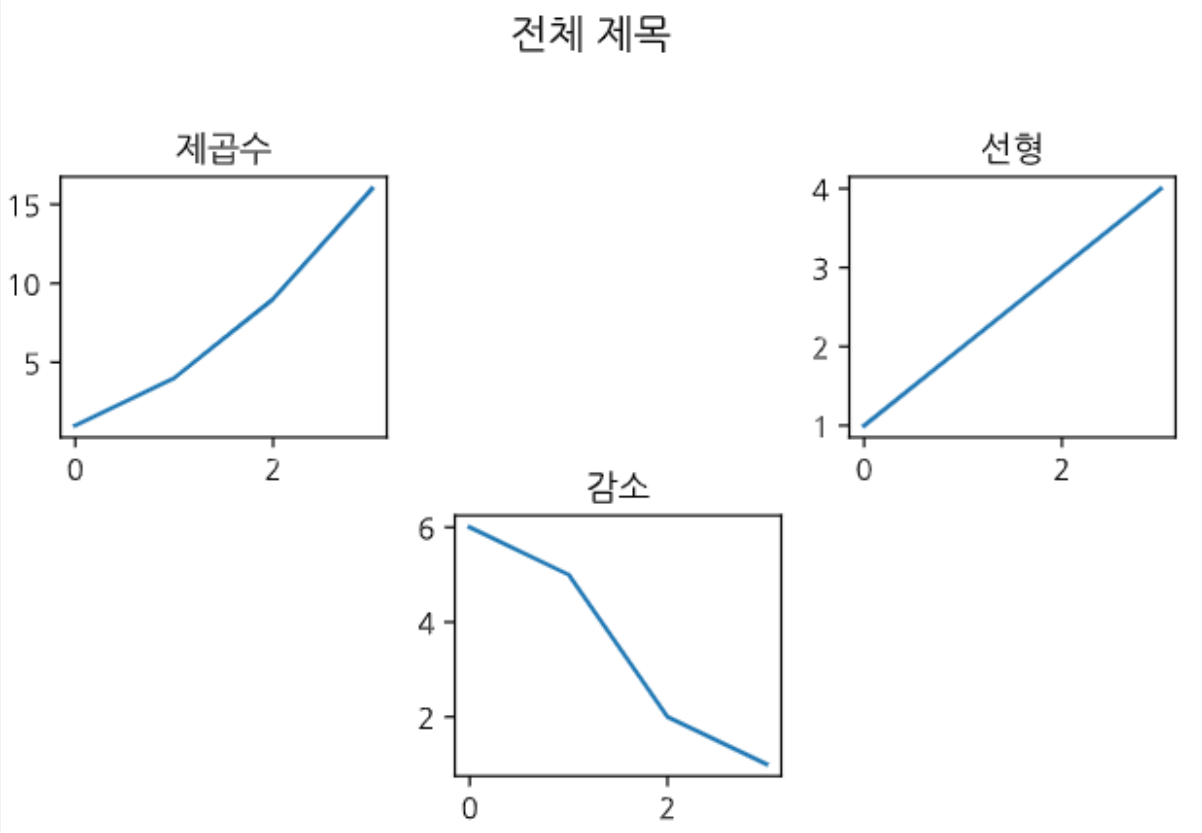
- plt.suptitle('전체제목')
- ax[m, n].set_title('부분제목')




```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(2, 3, figsize=(7, 4))
fig.set_size_inches(6, 3)

ax[0, 1].plot([1, 4, 9, 16])
ax[1, 0].plot([1, 2, 3, 4])
ax[1, 2].plot([6, 5, 2, 1])

# 전체 제목 – Figure 레벨
fig.suptitle('전체 제목', fontsize=14, fontweight='bold')

# 각 subplot 제목 – Axes 레벨
ax[0, 0].set_title('(0,0) 빈 플롯')
ax[0, 1].set_title('(0,1) 제곱수')
ax[0, 2].set_title('(0,2) 빈 플롯')
ax[1, 0].set_title('(1,0) 선형')
ax[1, 1].set_title('(1,1) 빈 플롯')
ax[1, 2].set_title('(1,2) 감소')

plt.tight_layout()
plt.show()
```

fig.add_subplot() 그림 제목

- fig.suptitle('전체제목')
- axi.set_title('부분제목')




```
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(6, 4))

ax1 = fig.add_subplot(231)
ax1.plot([1, 4, 9, 16])
ax1.set_title('제곱수')

ax2 = fig.add_subplot(233)
ax2.plot([1, 2, 3, 4])
ax2.set_title('선형')

ax3 = fig.add_subplot(235)
ax3.plot([6, 5, 2, 1])
ax3.set_title('감소')

fig.suptitle('전체 제목', fontsize=14)
plt.tight_layout()
plt.show()
```

plt.subplots()과 fig.add_subplot() 비교

- **plt.subplots()**
  - 간단한 다중 플롯 생성에 적합
  - 한 번에 figure와 axes 객체를 모두 생성해 간결한 코드 작성이 가능
- **fig.add_subplot()**
  - 개별적으로 Axes를 추가해야 하지만, 복잡한 레이아웃이나 더 세밀한 제어가 필요할 때 유용

특징	plt.subplots()	fig.add_subplot()
역할	figure와 axes 객체를 한 번에 생성	figure를 생성한 후 개별적으로 Axes 추가
figure 생성	자동으로 생성됨	plt.figure()로 먼저 생성해서 반환 받은 fig를 사용함
Axes 생성	한 번에 여러 Axes 객체를 배열 형태로 반환	각각의 Axes를 하나씩 추가
코드 간결성	더 간결함. 한 번의 호출로 여러 서브플롯 생성 가능	복잡해질 수 있음. 서브플롯을 하나씩 추가해야 함
figsize 설정	plt.subplots(figsize=(width, height))로 지정	plt.figure(figsize=(width, height))로 지정
반환 값	figure와 axes 배열을 동시에 반환	add_subplot() 호출 시 개별 Axes 반환
서브플롯 레이아웃	한 번에 설정 (nrows, ncols 인자 사용)	각 서브플롯을 개별적으로 추가 (add_subplot)
적합한 사용 사례	다수의 서브플롯을 그릴 때 빠르고 간결하게 사용	서브플롯을 개별적으로 세밀하게 제어할 때 유용
축 공유 기능	sharex, sharey 옵션으로 축 공유 가능	수동으로 설정해야 함
복잡한 레이아웃 제어	자동 배치. 복잡한 레이아웃에 덜 유연	복잡한 레이아웃에서 더 유연하게 제어 가능
사용 예시	fig, axes = plt.subplots(2, 2)	fig = plt.figure(); ax = fig.add_subplot(2, 2, 1)